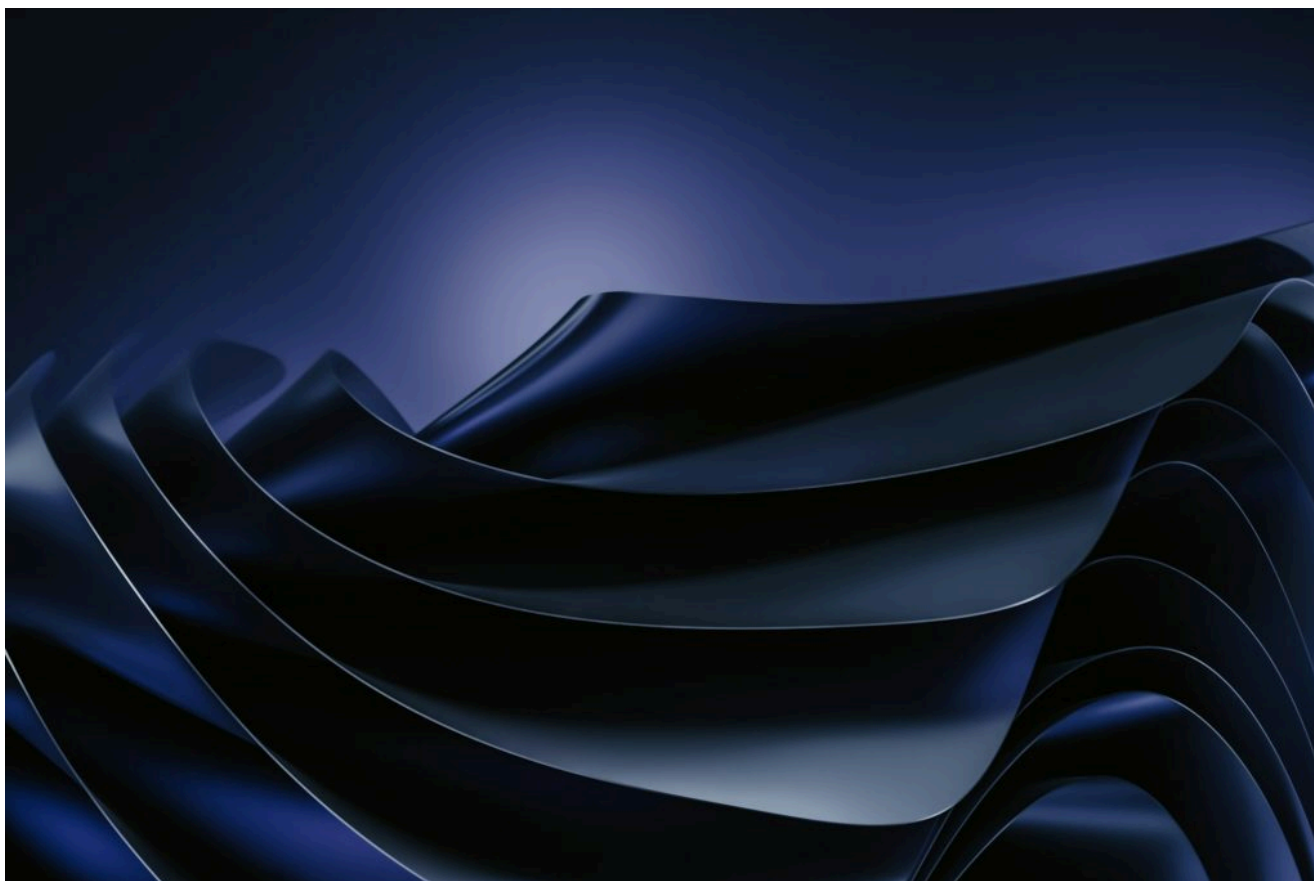


KUBERNETES / СЕТЕВЫЕ ТЕХНОЛОГИИ / РАЗРАБОТКА ПЛАТФОРМЫ

От замысла к реализации: опыт масштабного использования контроллеров Kubernetes

Узнайте, как Amazon EKS масштабирует контроллеры Kubernetes, управляет памятью состояний и обеспечивает корректное применение сетевых политик в больших масштабах.

17 июля 2026 года, 8:00 [Шри Саран Баладжи Веллор Раджакумар](#) и [Джайант Варавани](#)



Павел Червински для Unsplash+

[AWS Marketplace](#).

Контроллеры Kubernetes — это то, что делает декларативную модель платформы реальной. Они отслеживают состояние, приводят его в соответствие с замыслом и продолжают делать это по мере изменения темы. Однако в более крупных масштабах самые сложные проблемы связаны уже не с **написанием цикла согласования**. Они связаны с

представления о системе. В этом посте мы расскажем о том, чему научились, работая с двумя критически важными контроллерами в Amazon EKS в условиях масштабирования.

Kubernetes обеспечивает связь между подами, но детальное регулирование трафика оставляет на усмотрение базовой платформы. Внутри кластера намерения, например, кому разрешено взаимодействовать с кем, все равно должны быть преобразованы в правила, которые может обеспечить канал передачи данных. Для небольшого эксперимента открытость — удобный вариант. Но для производственного трафика это становится проблемой, которую нужно решить, и вы хотите один раз выразить свое намерение и сохранить его, пока поды создаются и уничтожаются в течение всего дня.

В Amazon EKS этот пробел устраняют два контроллера. Контроллер сетевой политики регулирует трафик внутри кластера, определяя, какие поды могут взаимодействовать друг с другом. Контроллер ресурсов VPC регулирует доступ выбранных подов к ресурсам AWS за пределами кластера, назначая им собственные группы безопасности AWS — тот же брандмауэр с отслеживанием состояния, который AWS уже использует для инстансов EC2.

Обычно у пода нет собственной группы безопасности, и он наследует группу безопасности узла. Это означает, что база данных RDS, которая хочет разрешить доступ к одному пода, должна разрешить входящий трафик из группы безопасности узла, что неявно разрешает доступ и ко всем остальным подам на этом узле. Под с собственной группой безопасности можно назвать напрямую, поэтому правило входящего трафика RDS разрешает доступ только к этой единственной рабочей нагрузке.

Оба являются обычными контроллерами Kubernetes. Они следят за кластером, хранят состояние в памяти и сопоставляют то, что они видят, с тем, что было объявлено. Именно это сопоставление позволяет вам один раз заявить о своих намерениях и быть уверенным, что они будут соблюдаться, даже если поды будут появляться и исчезать. EKS запускает эти контроллеры **в разных кластерах**, количество которых варьируется от нескольких до сотен тысяч узлов, и именно из-за этого разнообразия и был написан этот пост. В небольших масштабах легко упустить из виду объем памяти, занимаемый контроллером, корректность сопоставления и взаимодействие компонентов. В наличии тысяч узлов и модулей они становятся обязательными



AWS Marketplace helps companies of all sizes find, try, buy, deploy, and manage solutions from AWS Partners. Accelerate solution discovery, improve governance, enhance cost transparency, and reduce SaaS sprawl with centralized billing and management on AWS.

[Learn More →](#)

HEAR MORE FROM OUR SPONSOR

svo1975pvn1982@gmail.com

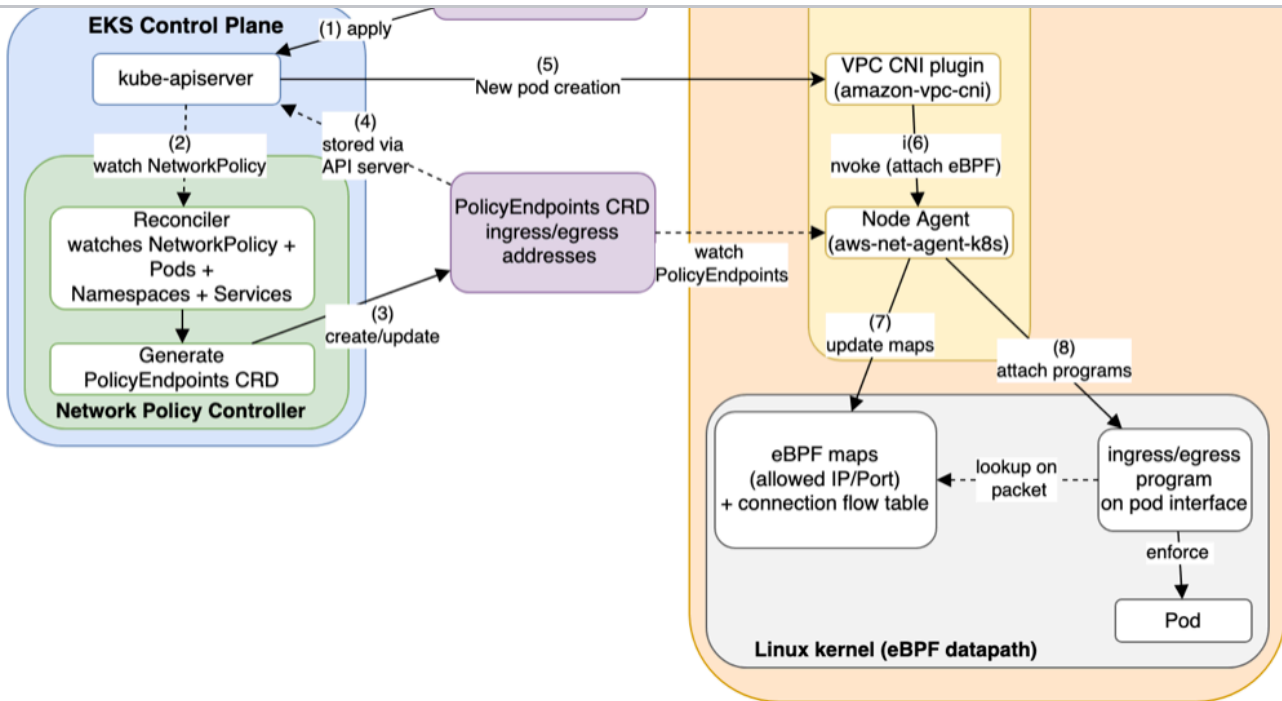
Submit

“At small scales, a controller’s memory footprint, reconciliation correctness, and component interactions are easy to miss. With thousands of nodes, they become binding constraints.”

This post looks at how these controllers work, what changes as they scale, and the patterns that emerged from operating them.

Two controller design choices that matter at scale

Network Policy Controller: How do you go from label selectors to packet-level enforcement?



TRENDING STORIES

1. [Kubernetes upgrades don't have to break things: How EKS is making cluster lifecycle management simpler and safer](#)
2. [From intent to enforcement: Lessons from operating Kubernetes controllers at scale](#)
3. [Operating Kubernetes at scale: a few stories from running Amazon EKS](#)
4. [Is Kubernetes Adoption Slowing?](#)
5. [What running Kubernetes across millions of clusters taught AWS about zonal failures](#)

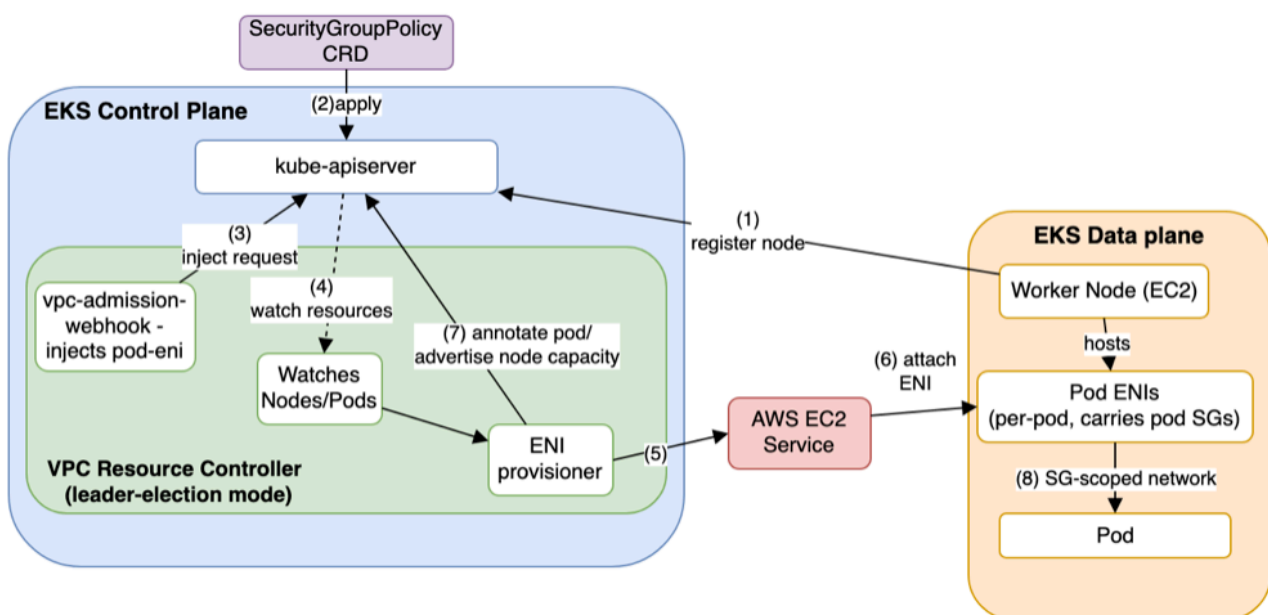
A NetworkPolicy is a declaration of intent written in labels. For example, pods labeled app=web may receive traffic only from pods labeled app=api. Labels are what make the rule durable, since they keep meaning the same thing as pods come and go, but labels are not what the datapath enforces. The eBPF program on each node matches on concrete pod IPs, so something has to resolve app=api into the current set of IPs and keep that set correct as pods churn.

own answer, and some network-policy implementations do exactly that. EKS does the join once, centrally. A reconciler in the Network Policy Controller monitors the NetworkPolicy objects, along with the Pods, Namespaces, and Services they reference, resolves selectors to IPs, and writes the resolved answer into a resource it owns, called a PolicyEndpoint.

When a policy matches many pods, or pods are constantly created and destroyed, the controller splits the result across several PolicyEndpoints rather than letting a single object grow too large or be rewritten too often. Each node's agent then acts only on the PolicyEndpoints for its own pods, rather than every node independently re-resolving the entire cluster. That is what keeps API server load manageable as the cluster grows.

From there, the work moves down to the worker nodes. The VPC CNI runs on each node inside the aws-node DaemonSet, and the same add-on runs a network policy agent beside it. The agent never writes back, which keeps the control plane and data plane cleanly separated. The controller decides, and the node enforces. Enforcement happens in the Linux kernel **using eBPF** rather than long chains of iptables rules. The agent programs the allowed addresses into the kernel as eBPF rules, and from then on, every packet entering or leaving the pod triggers a map lookup and is allowed or dropped right in the datapath.

VPC Resource Controller: How do you give a pod its own firewall identity?



the controller to honor that, the pod needs its own network interface, since a security group can only attach to an interface.

Each node has a limit on the number of pod network interfaces, and the limit depends on its instance type. An m5.large, for example, supports nine. The controller advertises that limit on each node. So, when you create a pod that matches the policy, the controller mutates the pod to add a request for one interface, and the scheduler treats it like any other resource it has to find room for, just as it looks for CPU or memory. The pod only lands on a node with a free network interface, and every pod that gets its own security group uses one up.

Once the pod is scheduled, the controller calls the EC2 API to assign it a network interface on the node, separate from the one the node uses for everything else. It applies the pod's security groups to that interface. Now, a rule on an RDS, ElastiCache, or OpenSearch resource can allow exactly that workload and nothing else to run on the same node.

What changes with thousands of nodes and pods

The cost of holding state

A controller's informer holds a local copy of every object it watches, so reconciliation never has to round-trip to the API server. By default, that copy is the full object: the entire pod spec, all containers, environment variables, annotations, and metadata. A single cached pod can be tens of kilobytes, and at tens of thousands of pods, that becomes hundreds of megabytes of fields the controller never reads. The VPC Resource Controller needs only certain fields such as the pod's node name, namespace, UID, and its own annotations. The Network Policy Controller needs only labels, pod IPs, container ports, and phase.

The VPC Resource Controller addressed this early by building a custom informer that strips each pod down to the fields it needs before caching it. The Network Policy Controller does the same thing in a different way, using a built-in Kubernetes hook that runs on every object as it arrives and keeps only the fields we need. It also filters out terminated and unscheduled pods on the server side, so they never reach the controller. On a large cluster, this cut memory from several hundred megabytes to under a hundred.

and test that reconciliation still works on the stripped-down objects.”

The pattern for your own controllers is simple. Define the fields your logic actually reads, strip everything else before it reaches the cache, and test that reconciliation still works on the stripped-down objects. Keep that trimming to field selection only, since it runs on every object one at a time, and an extra millisecond per object adds up fast on a large cluster.

When stale state can't self-correct

Kubernetes controllers are level-triggered and forgiving. Acting on a slightly stale view is usually safe, because the next reconcile corrects it. The exceptions are actions that cannot be undone, such as deleting a cloud resource, removing a network interface, or programming a rule to drop packets. For these, a stale view is not self-correcting, because there is no object left to reconcile against.

Before it acts, the controller must be certain its cache reflects reality across every object type the action depends on, not just some of them. The VPC Resource Controller periodically scans for interfaces that are no longer associated with any running pod and reclaims them. That decision compares two things: the existing interfaces and the pods currently running. In steady state, this is safe, since any interface it finds was provisioned by the controller itself, so the owning pod is already in its cache.

The hazard appears at startup. When the controller restarts or a new leader takes over, the interfaces created by the previous instance remain on the nodes, but the new instance starts with an empty cache. If its cleanup scan runs before the pod cache has finished loading, a pod that is genuinely running simply has not appeared yet, so the controller assumes it was deleted and reclaims an interface that live traffic is using.

The Network Policy Controller faces the same problem from the other direction. It enforces policies whose resolved addresses are split across several PolicyEndpoint objects, and it must hold enforcement until the complete set has arrived, because acting on a partial set can open traffic the policy was written to block. In both cases, the fix is the same. The controller waits until all its caches have finished loading from the API server before it runs the irreversible action.

on a partial state. The cost of getting this right is a few seconds at boot. The cost of skipping it is deleting infrastructure that is still in use.

When an optimization shifts the constraint

The VPC Resource Controller's pod interfaces carry a second-order effect that only surfaces at scale. Every pod with its own security group draws a single IP address from the subnet, and that address lands wherever a free slot happens to be in the provided subnet. On a busy cluster, these scattered addresses accumulate alongside ordinary secondary IPs that regular pods take and release as they churn, and over time, the subnet's free space grows fragmented. None of this matters until you enable an optimization that depends on contiguity.

That optimization was prefix delegation for regular pods. EKS normally hands each pod one secondary IP from the ENIs on the node, and because every address costs an EC2 API call, a node tends to exhaust its pod budget long before it runs out of CPU or memory. Prefix delegation changes the unit of request so the node asks for a /28 block of sixteen addresses at once, which lifts density to 110 pods on the same instance and cuts API traffic to a fraction of what it was.

The catch is that a /28 needs sixteen contiguous addresses. A fragmented subnet can report thousands of free IPs while still failing every prefix allocation with an `InsufficientCidrBlocks` error, because no sixteen of them sit next to each other. Pods stop scheduling even as every dashboard insists the subnet is healthy, because the monitoring built for the per-IP model counts free addresses, and a free-address count says nothing about whether those addresses are contiguous. The fix is to give prefixes their own contiguous space to draw from, either through VPC subnet CIDR reservations or a dedicated subnet.

The lesson generalizes. An optimization may not actually remove a constraint. It moves it somewhere new, and the metric that looked healthy under the old design can stay green long after the real limit has moved. When you make a change for density or throughput, look for where the pressure went, and make sure something you watch would catch it.

“An optimization may not actually remove a constraint. It moves it nowhere new, and the metric that looked healthy under the old

The controllers that turn network policy into eBPF rules and attach per-pod security groups are ordinary Kubernetes controllers. What lets them hold up at scale is not special infrastructure but deliberate design: cache only the state you actually read, wait for the state to be complete before taking irreversible action, and ensure your monitoring reflects the system's real constraints. Those lessons extend well beyond EKS. Any controller that translates high-level intent into concrete enforcement has to solve the same problems of state, timing, and scale.

To use these capabilities on your EKS cluster, read our AWS docs on [Network Policy](#) and [Security group for pods](#).

TNS



Sri Saran Balaji Vellore Rajakumar is a Principal Engineer at Amazon Web Services and a founding engineer of Amazon EKS, where he has spent nearly a decade building the infrastructure that keeps the EKS control plane available and performant at...

[Read more from Sri Saran Balaji Vellore Rajakumar →](#)



Jayanth Varavani is a Software Development Manager at Amazon Web Services, where he leads the EKS Node Runtime and EKS Networking teams that build and operate the core infrastructure powering Amazon EKS at scale. He and his teams own the...

[Read more from Jayanth Varavani →](#)

SHARE THIS STORY



TRENDING STORIES

1. [Kubernetes upgrades don't have to break things: How EKS is making cluster lifecycle management simpler and safer](#)
2. [From intent to enforcement: Lessons from operating Kubernetes controllers at scale](#)

5. What running Kubernetes across millions of clusters taught AWS about zonal failures

TNS DAILY NEWSLETTER

Receive a free roundup of the most recent TNS articles in your inbox each day.

SUBSCRIBE

The New Stack does not sell your information or share it with unaffiliated third parties. By continuing, you agree to our [Terms of Use](#) and [Privacy Policy](#).

ARCHITECTURE

Cloud Native Ecosystem
Containers
Databases
Edge Computing
Infrastructure as Code
JMX

ENGINEERING

AI
AI Engineering
API Management
Backend development
Data
Frontend Development

[Networking](#)[Storage](#)[Software Development](#)[WebAssembly](#)

OPERATIONS

[AI Operations](#)[CI/CD](#)[Cloud Services](#)[DevOps](#)[Kubernetes](#)[Observability](#)[Operations](#)[Platform Engineering](#)

CHANNELS

[Podcasts](#)[Ebooks](#)[Events](#)[Webinars](#)[Newsletter](#)[TNS RSS Feeds](#)

THE NEW STACK

[About / Contact](#)[Sponsors](#)[Advertise With Us](#)[Contributions](#) [roadmap.sh](#)

Community created roadmaps, articles, resources and journeys for developers to help you choose your path and grow in your career.

[Frontend Developer Roadmap](#)[Backend Developer Roadmap](#)[Devops Roadmap](#)

FOLLOW TNS



© The New Stack 2026

[Disclosures](#)[Terms of Use](#)[Advertising Terms & Conditions](#)[Privacy Policy](#)[Cookie Policy](#)

